

Actividad 10

Laboratorio: File Path Traversal, Traversal Sequences Stripped with Superfluous URL-decode

PortSwigger Web Security Academy

Elaborado por: Juan Pablo Casanova De La Madrid

Herramienta utilizada: Burp Suite Community Edition

1. Nombre del laboratorio

File path traversal, traversal sequences stripped with superfluous URL-decode

Laboratorio perteneciente a PortSwigger Web Security Academy, dentro de la categoría File path traversal.

2. Nivel

Practitioner.

Este laboratorio incorpora dos capas de procesamiento sobre la entrada del usuario: primero, la aplicación bloquea cualquier valor que contenga literalmente secuencias de traversal (".."); después, realiza una decodificación URL adicional (superflua) sobre el valor ya validado, antes de usarlo para acceder al sistema de archivos. Esta combinación exige una técnica de bypass distinta a las empleadas en los laboratorios anteriores (ruta absoluta y secuencias anidadas), por lo que se mantiene en el nivel Practitioner.

3. Tipo de explotación

Path Traversal / Directory Traversal mediante doble codificación URL (CWE-22: Improper Limitation of a Pathname to a Restricted Directory).

Es una vulnerabilidad del lado del servidor que permite a un atacante manipular la ruta de un archivo solicitado para acceder a archivos fuera del directorio raíz previsto por la aplicación. En este caso, la explotación requiere codificar el payload de traversal dos veces (superfluous / double URL-encoding), de manera que el filtro de entrada no detecte la secuencia ".." en su forma literal, pero esta se reconstruya al aplicarse una decodificación URL adicional por parte de la aplicación.

4. Objetivo del laboratorio

El laboratorio contiene una vulnerabilidad de path traversal en la funcionalidad que muestra las imágenes de los productos de la tienda. La aplicación bloquea cualquier entrada que contenga secuencias de path traversal y, a continuación, realiza una decodificación URL del valor recibido antes de utilizarlo.

Objetivo: explotar dicha vulnerabilidad codificando el payload de traversal mediante URL-encoding para evadir el filtro inicial, aprovechando que la decodificación adicional realizada por la aplicación revela la secuencia de traversal real, con el fin de recuperar el contenido del archivo del sistema /etc/passwd y dar por resuelto el laboratorio.

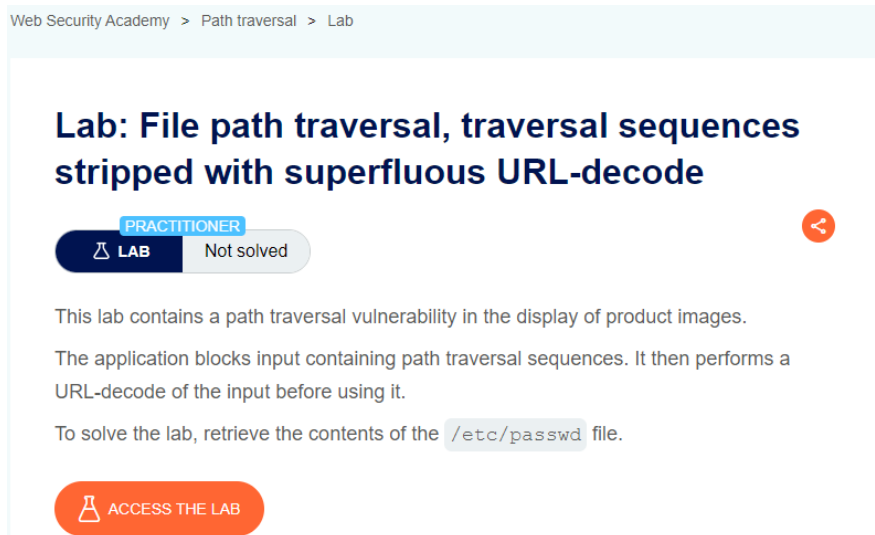


Figura 1. Descripción y objetivo del laboratorio en PortSwigger Web Security Academy (nivel Practitioner, estado Not solved).

5. Explicación técnica de la vulnerabilidad

La aplicación web muestra el catálogo de productos ("We Like To Shop") y cada imagen se sirve dinámicamente mediante un endpoint del tipo:

```
GET /image?filename=<nombre_de_archivo>
```

Como primera medida de protección, la aplicación examina el valor recibido en el parámetro filename y bloquea la petición si detecta literalmente la secuencia "../" (o variantes equivalentes). Este control se aplica sobre el valor tal como llega en la URL, es decir, antes de cualquier decodificación adicional.

El problema es que, después de pasar ese primer filtro, la aplicación realiza una decodificación URL adicional (superflua) sobre el valor ya validado, antes de usarlo para construir la ruta física del archivo. Esta segunda decodificación no estaba prevista como parte del control de seguridad, sino que forma parte del procesamiento normal de la aplicación, y es precisamente lo que introduce la vulnerabilidad.

Al codificar el carácter "/" en su forma URL-encoded (%2f) y, a su vez, codificar el símbolo de porcentaje de esa codificación (%25), se obtiene la secuencia %252f. Cuando el servidor recibe la petición, su capa de parsing HTTP decodifica una vez el valor (%25 → %), quedando en el parámetro la cadena ../%2f, que ya no contiene el patrón literal "../" y por lo tanto supera el filtro anti-traversal. Posteriormente, cuando la aplicación realiza su decodificación URL adicional sobre ese valor ya filtrado, %2f se convierte en /, reconstruyendo la secuencia ../ y permitiendo el recorrido fuera del directorio de imágenes. Esta vulnerabilidad se clasifica dentro de CWE-22 y ejemplifica los riesgos de aplicar más de una decodificación sobre una misma entrada sin revalidar el resultado final.

6. Procedimiento paso a paso

1. Se inicia Burp Suite Community Edition, seleccionando "Temporary project in memory" y la configuración por defecto ("Use Burp defaults").

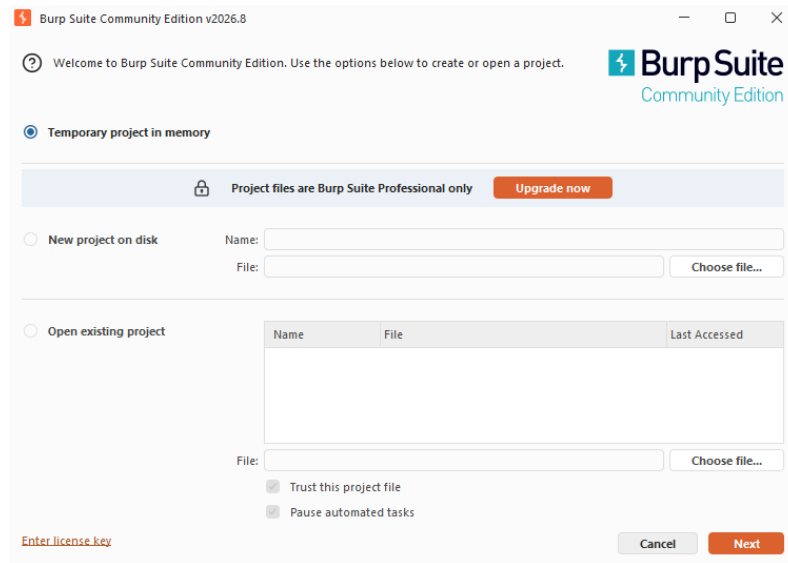


Figura 2. Creación de un proyecto temporal en memoria.

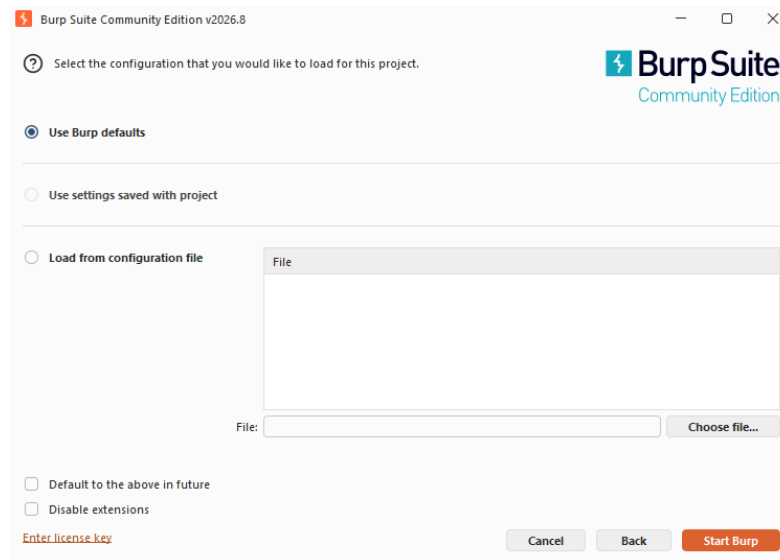


Figura 3. Selección de la configuración por defecto de Burp.

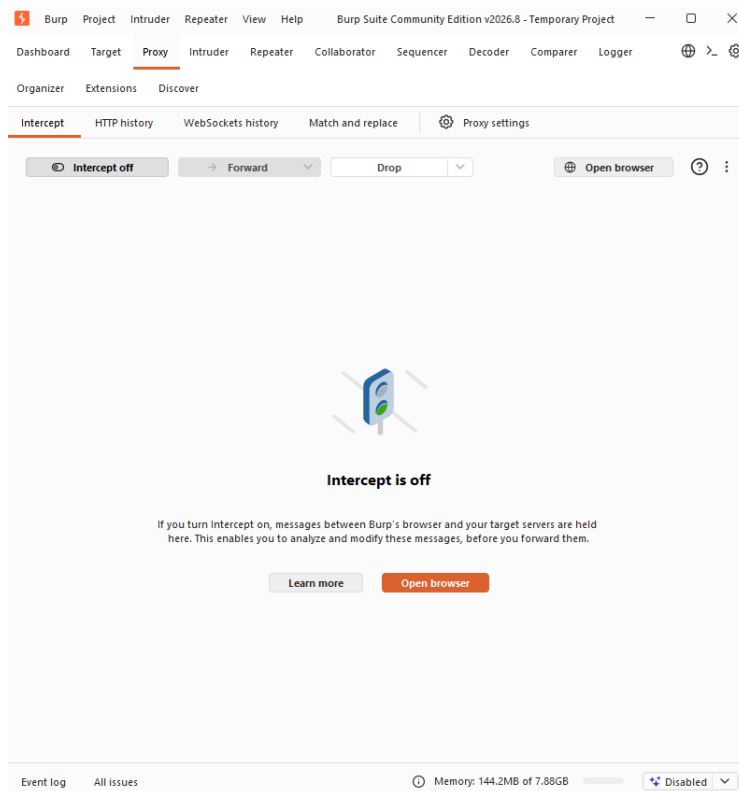


Figura 4. Burp Suite iniciado, pestaña Proxy con Intercept inicialmente apagado.

- Se accede al laboratorio desde la Web Security Academy de PortSwigger, con el tráfico pasando a través del navegador integrado de Burp.

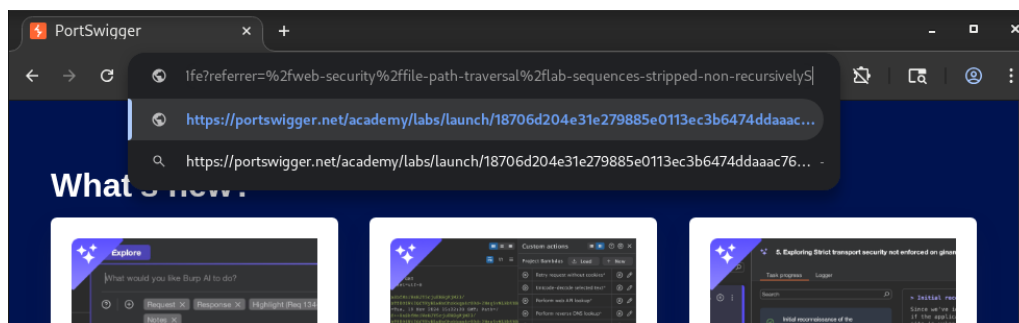


Figura 5. Navegación hacia la URL del laboratorio a través del navegador de Burp.

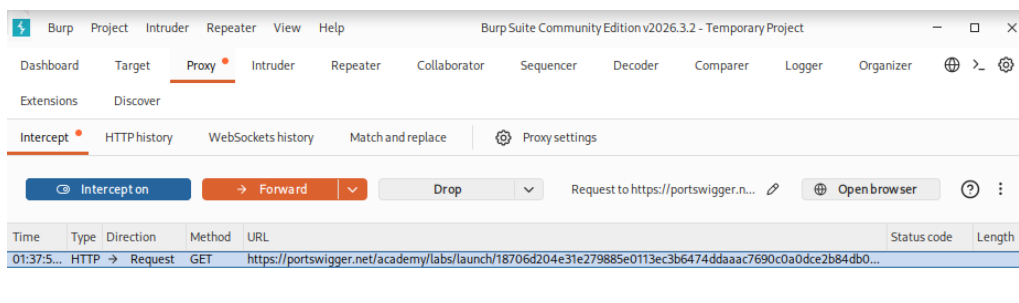


Figura 6. Petición GET inicial capturada en el historial del Proxy al abrir el laboratorio.

3. Con el Intercept activo, se navega por la tienda; Burp captura las peticiones GET que la página realiza para cargar cada imagen de producto, del tipo /image?filename=NN.jpg.

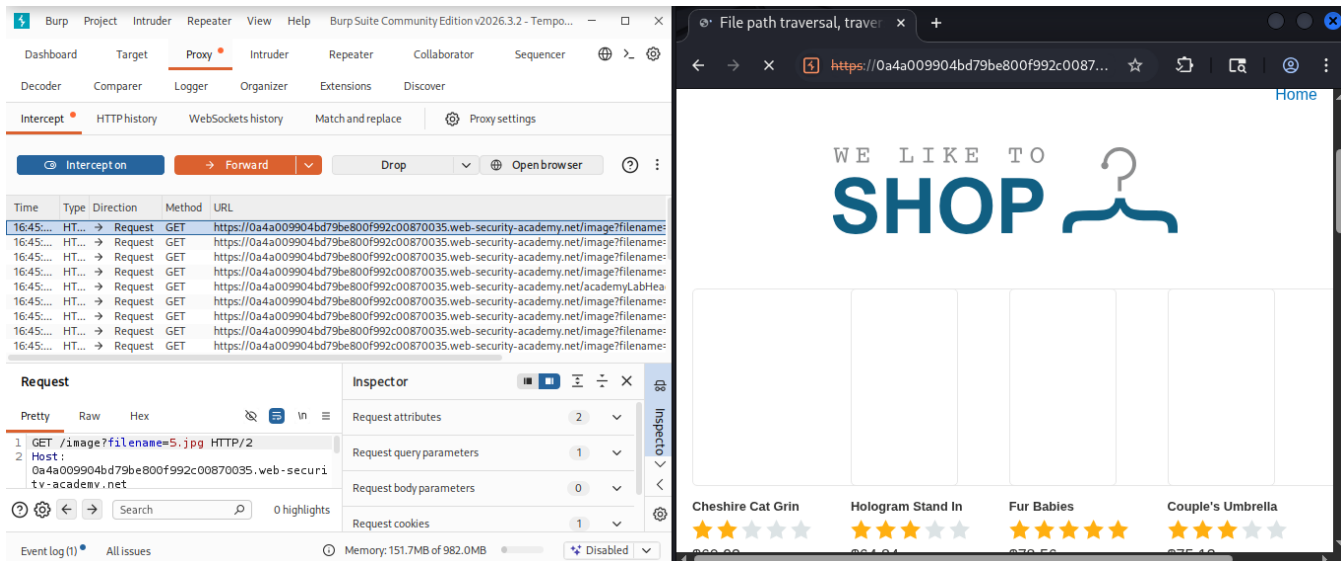


Figura 7. Tienda cargada e historial del Proxy mostrando las peticiones de imágenes interceptadas.

4. Una vez que carga la página, se da clic en "View details" de cualquiera de los productos, con el fin de capturar el método GET correspondiente y poder modificar sus parámetros.

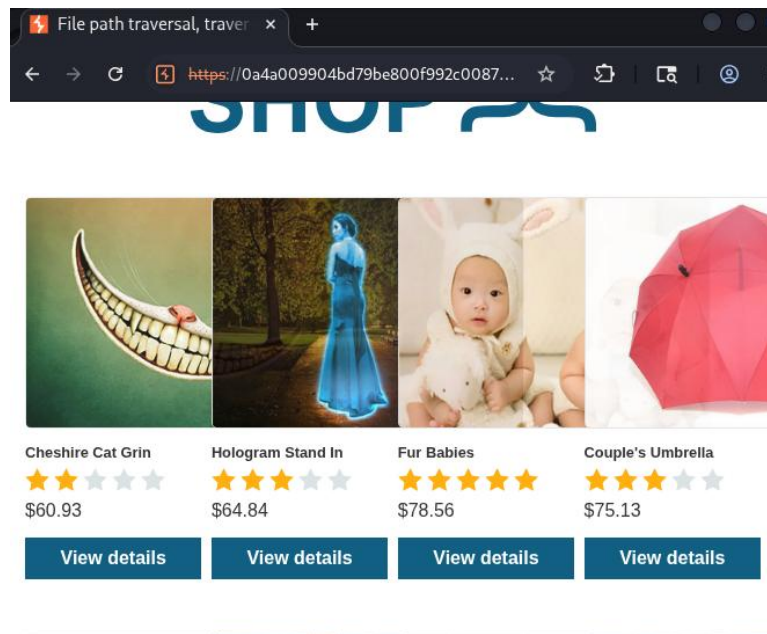


Figura 8. Vista de la tienda; se selecciona un producto mediante "View details" para capturar la petición.

5. Se identifica la petición GET /image?filename=5.jpg interceptada y, mediante clic derecho, se envía al módulo Repeater ("Send to Repeater") para poder manipular el parámetro filename.

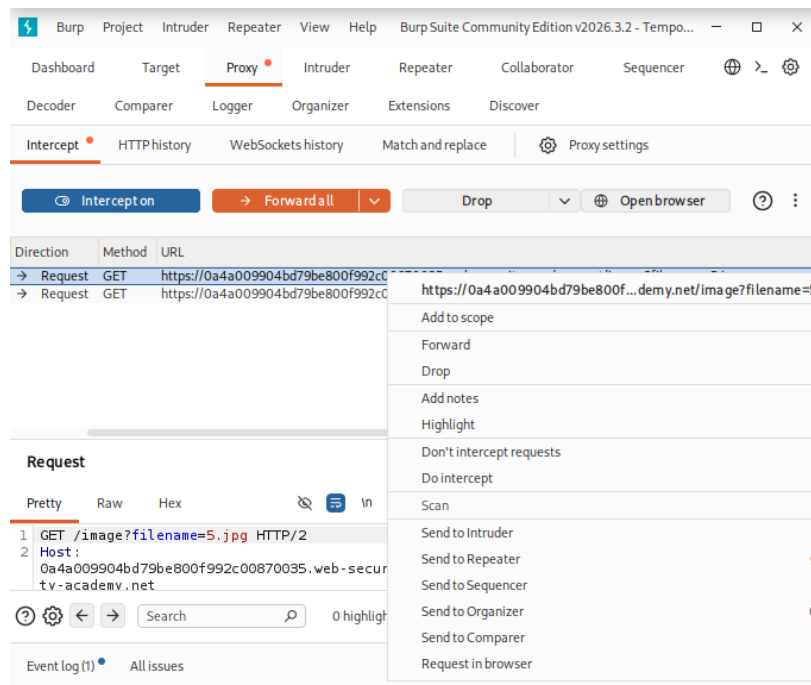


Figura 9. Menú contextual: envío de la petición GET /image?filename=5.jpg a Repeater.

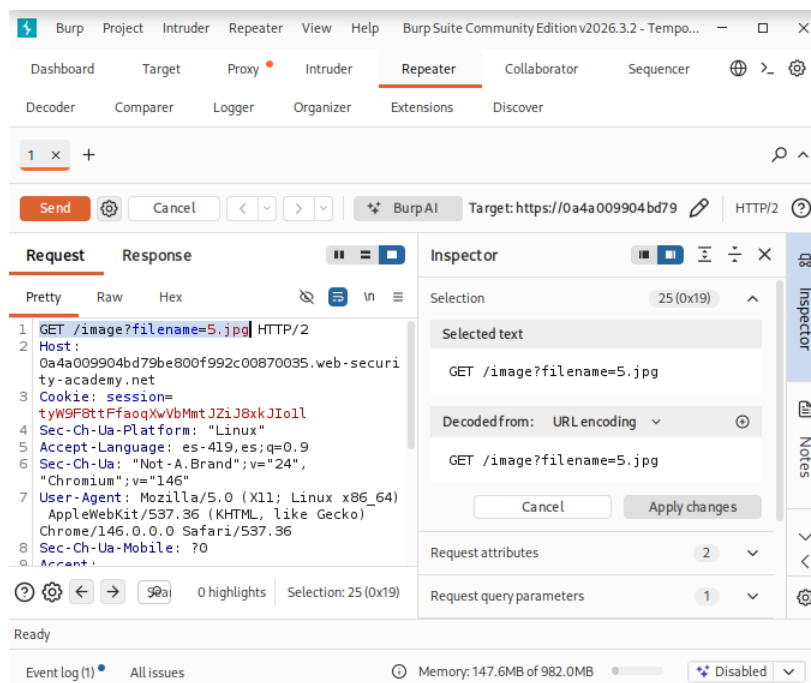


Figura 10. Petición cargada en Repeater; se inspecciona el valor filename=5.jpg antes de modificarlo.

6. En Repeater, dentro de GET /image?filename=5.jpg, se sustituye 5.jpg por el payload de doble codificación URL (superfluous URL-decode), diseñado para que el servidor no detecte ni elimine la secuencia de traversal al validarla.

GET /image?filename=..%252f..%252f..%252fetc/passwd HTTP/2

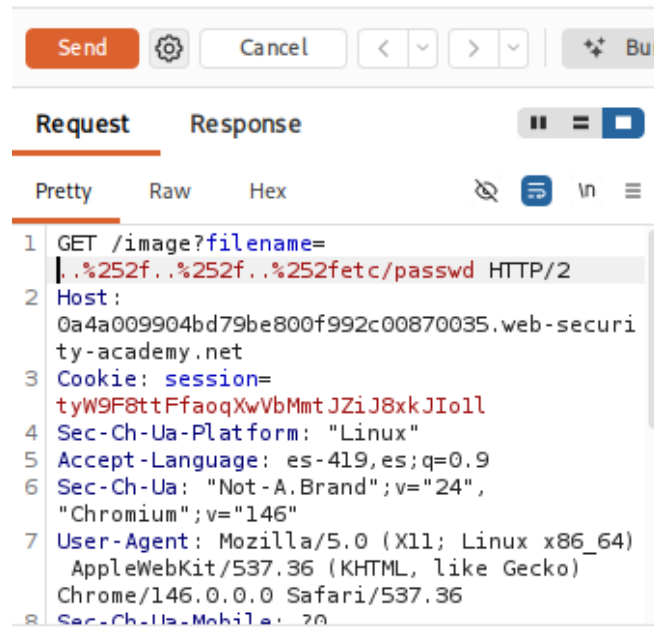


Figura 11. Payload de doble codificación URL inyectado en el parámetro filename.

7. Se hace clic en Send y se analiza la respuesta del servidor.
8. El servidor responde con código HTTP/2 200 OK y devuelve en el cuerpo de la respuesta el contenido íntegro del archivo /etc/passwd, confirmando la explotación exitosa de la vulnerabilidad.

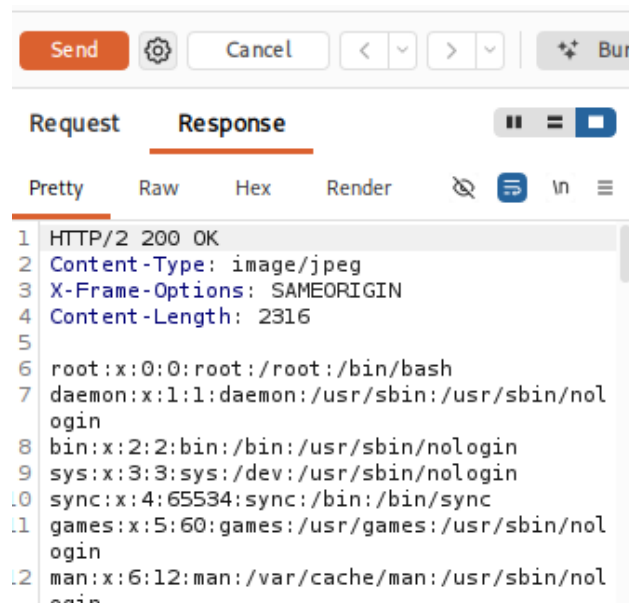


Figura 12. Respuesta vulnerable del servidor (200 OK) mostrando el contenido de /etc/passwd.

9. Finalmente, se regresa a la página del laboratorio en PortSwigger, la cual actualiza automáticamente su estado a "Solved", confirmando que el objetivo de la actividad fue completado.



Figura 13. Confirmación de laboratorio resuelto ("Solved").

7. Explicación del payload

GET /image?filename=..%25f..%25f..%25fetc/passwd HTTP/2

El payload se construye aprovechando dos niveles de codificación URL superpuestos:

- **%252f**: es la codificación URL del carácter "%" (%25) seguida de "2f", es decir, es el carácter "/" (%2f) codificado una segunda vez. Al llegar la petición al servidor, la capa HTTP realiza la decodificación URL estándar una sola vez, convirtiendo %25 en %, por lo que el valor que efectivamente recibe el filtro de la aplicación es ..%2f, no ../.
- **Evasión del filtro inicial**: como el filtro anti-traversal busca la secuencia literal "../", y en este punto el parámetro contiene "..%2f" (con el símbolo % en lugar de /), la validación se supera sin detectar el intento de traversal.
- **Decodificación superflua de la aplicación**: después de superar el filtro, la aplicación aplica una segunda decodificación URL sobre el valor ya validado. En este paso, %2f se convierte en /, reconstruyendo la secuencia ../ tres veces y permitiendo retroceder desde el directorio de imágenes hasta la raíz del sistema de archivos, donde se referencia etc/passwd.

En síntesis, el payload no oculta la intención de traversal de forma permanente, sino que la disfraza únicamente durante el momento en que el filtro de seguridad la examina, apoyándose en un paso de decodificación adicional que la propia aplicación ejecuta después de validar la entrada.

8. Mitigación recomendada

Para prevenir vulnerabilidades de path traversal como la explotada en este laboratorio, se recomienda aplicar las siguientes medidas, de forma combinada:

- Aplicar la validación de entrada después de realizar todas las decodificaciones necesarias (incluida cualquier decodificación URL adicional), nunca el filtro debe evaluar el valor final que realmente se usará para acceder al sistema de archivos.
- Evitar decodificaciones múltiples o redundantes sobre un mismo dato; si la entrada ya fue decodificada por la capa HTTP del servidor, no debe volver a decodificarse manualmente dentro de la lógica de la aplicación.
- Validar el parámetro filename contra una lista blanca (whitelist) de nombres de archivo o identificadores permitidos, en lugar de depender de la detección de patrones maliciosos en cualquier etapa del procesamiento.
- Resolver la ruta final de forma canónica (canonicalización) y verificar que el resultado permanezca dentro del directorio base autorizado, como última comprobación antes de acceder al archivo.
- Evitar exponer directamente el sistema de archivos del servidor a partir de datos proporcionados por el usuario; en su lugar, mapear identificadores de producto/imagen a rutas físicas mediante una tabla o índice controlado por el backend.
- Ejecutar el proceso de la aplicación con el mínimo privilegio necesario, restringiendo los permisos de lectura del usuario del servicio web únicamente a los directorios estrictamente necesarios.
- Reforzar la defensa en profundidad con un WAF (Web Application Firewall) que normalice y decodifique completamente las peticiones antes de aplicar sus reglas de detección de path traversal, y aplicar monitoreo/alertas sobre accesos a archivos fuera de lo esperado.